

The output of `VACUUM FULL` is pretty much the same as plain `VACUUM`: it shows that 1 million tuples can be thrown away. The end result, however, is that the whole table has gained the space occupied by said tuples. It is important to remember, however, that, while tempting, `VACUUM FULL` forces a complete table rewrite and, therefore, pushes a lot of work down to the I/O system, thus incurring potential performance penalties.

It is possible to summarize the main effects of `VACUUM` in pictures. Imagine a situation like the one depicted in the following figure, where a table occupies two data pages, with four and three valid tuples (the green ones), respectively:

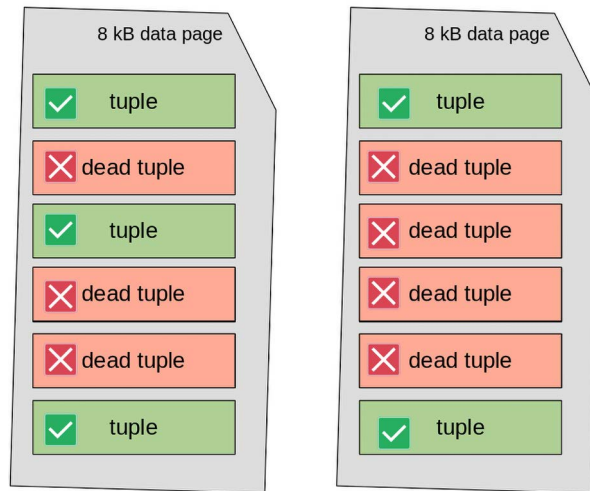


Figure 11.6: Valid and invalid tuples on two data pages

Dead tuples (the red ones) produce intra-page fragmentation, since they are interleaved with visible tuples. The final effect is that the table occupies storage space for two pages, while all the visible tuples could be “packed” into a single page.

If plain `VACUUM` executes, the total number of pages will remain the same, but every page will free the space occupied by dead tuples and compact valid tuples together, as shown in the following figure:

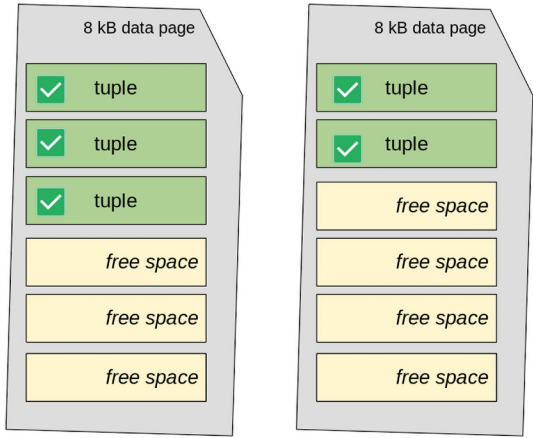


Figure 11.7: The results of a plain VACUUM

If `VACUUM FULL` executes, the table’s data pages are fully rewritten to compact all valid tuples together. In this situation, the second page of the table results is empty and, therefore, can be discarded, freeing up storage space. The situation becomes the one depicted in the following diagram:

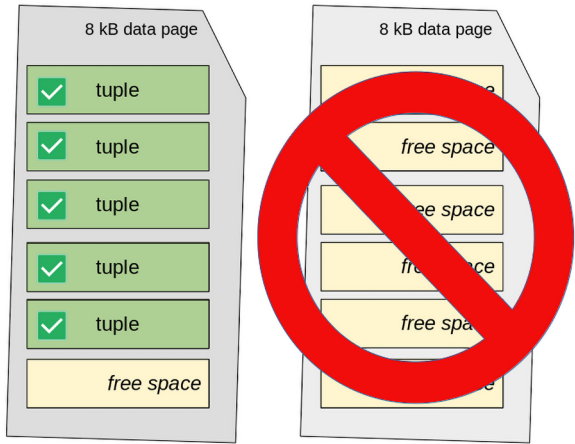


Figure 11.8: The results of a FULL VACUUM

It should be clear now what the main difference between plain and FULL VACUUM is: as a rule of thumb, plain VACUUM does not free storage space and is much less aggressive than VACUUM FULL, which, conversely, frees disk space. There is only a particular situation where plain VACUUM can give back a tiny portion of storage space: if all the tuples on the last page are dead, the page itself is deallocated.

Usually, you do not run `VACUUM` by hand, since PostgreSQL provides a much better approach to keep fragmentation under control by means of *automatic vacuuming*, explained in the next section.

Automatic VACUUM

Since PostgreSQL 8.4, there has been a background job named `autovacuum`, which is responsible for running `VACUUM` on behalf of the system administrator.

The idea is that, with `VACUUM` being an I/O-intensive operation, a background job can perform small micro-vacuums without interfering with the normal database activity.

Usually, you don't have to worry about `autovacuum`, since it is enabled by default and has general settings that can be useful in many scenarios; however, like pretty much everything in PostgreSQL, you can use specific settings to fine-tune the behavior of automatic vacuuming. A system with a good `autovacuum` configuration usually does not need manual `VACUUM`, and often, the traits of manual `VACUUM` mean that `autovacuum` must be configured to run more frequently.

Automatic vacuum works with a bunch of background processes, called *autovacuum workers*. Every worker is assigned to a database to work on; once the process has completed its activity, it terminates. PostgreSQL routinely starts new `autovacuum` worker processes, so that every database (and table) in the cluster gets a chance to get vacuumed automatically. However, PostgreSQL allows the database administrator to carefully set the behavior of this process-spawning activity: the `autovacuum_max_workers` configuration settings dictate the maximum number of active processes that can be running at a given time.

The `autovacuum` worker performs three main activities:

- Executes a plain `VACUUM` on data tables, in order to reduce fragmentation and continuously allocate new space to handle new tuple versions.
- Updates the system statistics about the quantity and quality of data stored in user tables, like a manual `ANALYZE` would do. This is very useful in order to let the query executor decide the best plan to access data, such as executing a *tuple freeze* whenever it is necessary, therefore preventing the problem of the `xid` wraparound.
- Executes a tuple freeze whenever it is necessary, therefore preventing the problem of the `xid` wraparound.

`autovacuum` is turned on by default, but you can always choose to disable it, even if this does not make any sense; usually, you need `autovacuum` to run more, not less. It is, however, important to keep in mind that, even when turned off, an emergency `autovacuum` process could start in order to prevent the `xid` wraparound problem.

In other words, PostgreSQL tries very hard to stay operative even if you misconfigure it!

The main settings for autovacuum can be inspected from the `$PGDATA/postgresql.conf` configuration file or, as usual, the `pg_settings` catalog. The most important configuration parameters are the following:

- `autovacuum` enables or disables the autovacuum background machinery. There is no reason, beyond doing experiments, as we did in the previous section, to keep autovacuum disabled.
- `autovacuum_vacuum_threshold` indicates how many new tuple versions will be allowed before autovacuum can be activated on a table. The idea is that we don't want autovacuum to trigger if only a small number of tuples have changed in a table, because that will produce an I/O penalty without an effective gain. By default, this parameter is set to 50 tuples, meaning that any change in your tables that does not produce at least 50 new tuple versions will not be considered sufficient to trigger autovacuum.
- `autovacuum_vacuum_scale_factor` indicates the amount, as a percentage, of tuples that have to be changed before autovacuum performs a concrete VACUUM on a table. The idea is that the more the table grows, the more autovacuum will wait for dead tuples before it performs its activities. This setting is, on a default installation, 0.2, meaning autovacuum will trigger once at least 20% of the tuples have been marked as dead.
- `autovacuum_cost_limit` is a value that measures the maximum threshold over which the background process must suspend itself to resume later on.
- `autovacuum_cost_delay` indicates how many milliseconds (in multiples of ten) autovacuum will be suspended to not interfere with other database activities. The suspension is performed only when the cost delay is reached.

Essentially, the activity of autovacuum goes like this: it scans every table within a database, and if the number of changed tuples is greater than $\text{autovacuum_vacuum_threshold} + (\text{table-tuples} * \text{autovacuum_vacuum_scale_factor})$, the autovacuum process activates. It then performs a vacuum on the table measuring the amount of work. If the amount of work reaches what `autovacuum_cost_limit` is set to, the process suspends itself for `autovacuum_cost_delay` milliseconds, and then resumes and proceeds further. Any time autovacuum reaches the threshold, it suspends itself, producing the effect of an incremental VACUUM. This *stop-and-go* behavior of autovacuum is intended to reduce the overall impact on the running cluster: autovacuum will suspend itself in order to leave resources available to interactive connections and users.

But how does autovacuum compute the cost of the activity it is doing? There is a set of tunable values that express how much it costs to fetch a new data page, scan a dirty page, and so on.

Such values are shared with manual VACUUM:

```
forumdb=> SELECT name, setting FROM pg_settings
           WHERE name like 'vacuum_cost%';
           name          | setting
-----+-----
vacuum_cost_delay       | 0
vacuum_cost_limit       | 200
vacuum_cost_page_dirty  | 20
vacuum_cost_page_hit    | 1
vacuum_cost_page_miss   | 2
(5 rows)
```

Such values are used for both manual VACUUM and autovacuum, with the exception that autovacuum has its own `autovacuum_vacuum_cost_limit`, which is usually set to 200. On the other hand, manual VACUUM has `vacuum_cost_delay` set to 0, meaning essentially that a manual VACUUM process will never suspend itself. After all, the database administrator wants the manual VACUUM to finish as soon as possible.

Similar parameters exist for the ANALYZE part because the autovacuum background process performs VACUUM ANALYZE, and therefore, you have `autovacuum_analyze_threshold` and `autovacuum_analyze_scale_factor`, which are in charge of defining the window of activity for the ANALYZE part (which is involved in updating the statistics on the contents of the table).

Summary

PostgreSQL exploits MVCC to provide high concurrent access to underlying data, and this means that every transaction perceives a snapshot of the data while the system keeps different versions of the same tuples. Sooner or later, invalid tuples will be removed, and storage space will be reclaimed. On one hand, MVCC provides better concurrency, but on the other hand, it requires extra effort to reclaim the storage space once transactions no longer reference dead tuples. PostgreSQL provides VACUUM for this aim and also has a background process machinery, named autovacuum, to periodically and non-invasively keep a system clean and healthy.

In order to improve I/O and reliability, PostgreSQL stores data in a journal written sequentially, the WAL. The WAL is split into segments, and at particular time intervals, named checkpoints, all the dirty data in memory is forced to a specified position in the storage, and the WAL segments are recycled.

In this chapter, you have learned about WAL and MVCC internals, as well as transaction boundaries and savepoints. You have also seen how to impose a specific transaction isolation level that, depending on your needs, can protect your data against concurrent updates of the same tuples.

In the next chapter, you will discover how PostgreSQL can be extended beyond its normal functionalities by means of pluggable modules, named extensions.

Verify your knowledge

- What is a transaction?

A transaction is a unit of work that is either consolidated or discarded as a whole. A transaction can be made by a single statement or multiple statements and can be implicit or explicit. See the *Introducing transactions* section for more details.

- What is an `xid` and to which problem is it subject?

An `xid` is a transaction identifier, a number that uniquely represents a transaction within the whole cluster. Being stored as a counter, the value is subject to the so-called problem of *xid wraparound*, which `VACUUM` and autovacuum freezing solve. See the *More about transaction identifiers – the XID wraparound problem* section for more details.

- What is MVCC?

MVCC is a technique by which, at a given instant, multiple versions of a tuple can exist within a database. Depending on the currently running transactions and their commit status, a different version is used. See the *Explaining MVCC* section for more details.

- What are WALs and why are they so important?

WALs are the *intent logs* of a database; before doing any modification to the storage, PostgreSQL writes changes to the WALs. This way, writes will be faster, and no data will be lost in the case of a crash. See the *How PostgreSQL handles persistency and consistency: WALs* section for more details.

- What is a checkpoint?

A checkpoint is a point in time where the cluster synchronizes the in-memory data with the on-disk data, ensuring the storage reflects the latest changes. After the completion of a checkpoint, old WAL segments can be recycled. See the *Checkpoints* section for more details.

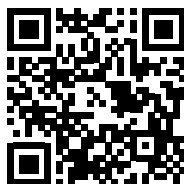
References

- PostgreSQL transaction isolation levels – official documentation: <https://www.postgresql.org/docs/current/sql-set-transaction.html>
- PostgreSQL transaction isolation level `SERIALIZABLE` – official documentation: <https://www.postgresql.org/docs/current/transaction-iso.html#XACT-SERIALIZABLE>
- PostgreSQL savepoints – official documentation: <https://www.postgresql.org/docs/current/sql-savepoint.html>
- PostgreSQL `VACUUM` – official documentation: <https://www.postgresql.org/docs/current/sql-vacuum.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



12

Extending the Database – the Extension Ecosystem

Extensions are a powerful way of packaging together related database objects, such as functions, routines, and tables, making the management of the objects as a single unit easier. Extensions allow you and other developers to extend the already rich PostgreSQL set of features by providing a clear, concise, and accurate way of installing, upgrading, and removing features and objects. In this chapter, you will see what extensions are and how they can be installed, upgraded, or removed with different tools. Moreover, you will learn how to build your own extension from scratch so that you will be immediately productive in packaging your own scripts and tools to distribute across other databases and PostgreSQL instances.

The chapter consists of the following topics:

- Introducing extensions
- Managing extensions
- Exploring the PGXN client
- Installing extensions
- Creating your own extension

Technical requirements

The chapter examples can be run on the `chapter_12` Docker image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage instructions for the Docker images for this book, please refer to *Chapter 1, Introduction to PostgreSQL*.

Introducing extensions

SQL is a declarative language that allows you to create and manipulate objects, as well as data. You can group SQL statements into scripts so that you can run the scripts in a more predictable and reproducible way. However, such scripts are seen by PostgreSQL as a sequence of unrelated commands, that is, you are responsible for correlating such commands into appropriate scripts. Things get even worse when you have to deal with *foreign programming languages* (e.g., PL/Perl and other not SQL-based languages) or binary libraries; the cluster knows nothing about your aims or how the objects are related to each other. Luckily, extensions help you get order out of chaos.

An extension is a packaged set of files that can be installed in the cluster in order to provide more functionalities, that is, to “extend” the current cluster set of features. Unlike scripts, extensions are managed strictly through specific commands that install, deploy, load, and upgrade the extension as a whole thing, even if it is made up of several different files.

An extension can be something general, like a new data type, a new index type, or a service to send emails directly from within PostgreSQL, or it can be something really specific to a particular use case, like a set of tables and data to provide ad hoc configuration. An extension does not have an opinion on how you are going to use it, and therefore you are free to install and forget it or use it in every database of your cluster.

The main aim of the extension mechanism is to provide a common interface for administering new features. Thanks to extensions, you have a common set of statements to deploy, install, upgrade, and remove an extension as a whole thing within the cluster. It does not matter whether your extension is made up of a single function or a whole set of linked objects, the extension mechanism will handle all the objects at once, making the administration easier. Moreover, the extension machinery defines a standard way to add features in a well-structured and clean way so that it becomes easier for everyone to contribute to PostgreSQL.

PostgreSQL comes with a set of useful extensions contained in the `contrib` package and developed by the PostgreSQL developers themselves. The `contrib` set of extensions are, therefore, solid and secure to use since they are tightly built with the PostgreSQL database itself. However, extensions can come from third parties, and this is the beauty of this approach: everyone can contribute to PostgreSQL by providing new features through extensions. Note that the PostgreSQL developers do not guarantee the stability of third-party extensions.



Operating system packages usually provide a `postgresql-contrib` package that installs all the PostgreSQL `contrib` extensions. This package is kept separated in order to let the user choose if those extensions have to be installed or not. Clearly, PostgreSQL works fine even without the `contrib` module, which in fact adds features by means of extensions.

PostgreSQL has built a whole ecosystem around the concept of extensions, and therefore not only does it provide statements for managing extensions, but also a platform for building new extensions and converting existing scripts into extensions. Then, extensions can be made publicly available by means of a global repository known as the **PostgreSQL eXtensions Network (PGXN)**.

You can think of PostgreSQL extensions as being reusable libraries in programming languages, such as modules for Perl, gems for Ruby, JARs for Java, and so on. Similarly, the PGXN infrastructure can be thought of as the CPAN to Perl (or PEAR to PHP, and so on).

The extension ecosystem

The beauty of extensions is that they provide a uniform way to bundle modules that can be deployed (installed) and used in PostgreSQL. Developers are free to contribute to expanding the number of modules available for PostgreSQL, and this has rapidly grown to what is now a full ecosystem.

Similar to programming languages, like Perl, Python, and others, PostgreSQL can now be customized with add-ons and modules that share a common infrastructure and architecture and can be managed by the same statements without any regard for the features they provide.

Extensions are mainly collected in the PGXN, an online repository that can be queried to get information about an extension or to download an extension (and a particular version of it), and can be updated with new modules.



Remember, the PGXN is like CPAN to Perl, CTAN to LaTeX, PEAR to PHP, and so on.